Chương 5 – Phần 2
Điều khiển Giao dịch đồng thời

GVHD: TS. Phan Thị Thể

FIT – HCM-UTE

Email: thept@hcmute.edu.vn

Nội dung chương 5 – Phần 2

1. Kĩ thuật khóa.
2. Khóa 2 pha.
3. Deadlock và Starvation.
4. Kĩ thuật nhãn thời gian.
5. Kĩ thuật sử dụng nhiều phiên bản.
6. Bài tập.

1. Kĩ thuật khóa.

1. Kĩ thuật khóa.

- **Giới thiệu**
  - Tìm hiểu một số kĩ thuật điều khiển đồng thời(Concurrency control) được sử dụng trong việc đảm bảo tính cô lập của các giao dịch được thực hiện.
  - Các kĩ thuật này đảm bảo tính khả tuần tự của lịch trình dựa trên các giao thức điều khiển đồng thời– Concurrency control protocols (protocols – sets of rules).

1. Kĩ thuật khóa.

- **Giao thức dựa trên khóa**
  - Một phương pháp để đảm bảo tính tuần tự dựa trên khái niệm khóa (LOCKING) các hạng mục dữ liệu.
  - Kĩ thuật khóa ngăn chặn nhiều giao dịch truy xuất 1 hạng mục dữ liệu trong cùng 1 thời điểm.
  - Cơ chế khóa được sử dụng trong hầu hết các hệ quản trị CSDL thương mại.
  - Yêu cầu việc truy xuất đến một hạng mục dữ liệu được tiến hành theo kiểu loại trừ lẫn nhau (mutual exclusion).
  - Một giao dịch đang truy xuất 1 hạng mục dữ liệu thì không cho phép giao dịch khác chỉnh sửa dữ liệu này.
  - Một khóa (lock) là một biến tương ứng với một hạng mục dữ liệu, quy định những hành động cụ thể nào được phép thực hiện trên hạng mục dữ liệu đó.
  - Thông thường: 1 khóa cho mỗi hạng mục dữ liệu.
  - Có nhiều loại khóa được sử dụng trong điều khiển song hành.

1. Kĩ thuật khóa.

- **Khóa nhị phân (Binary Lock)**
  - Đơn giản nhưng rất hạn chế nên không dùng trong thực tế.
  - 1 khóa nhị phân (binary lock) gồm 2 trạng thái:
    - Locked (1)
    - Unlocked (0).
  - Các khóa khác nhau trên mỗi hạng mục dữ liệu khác nhau.
  - Nếu trạng thái khóa của X là 1, hạng mục dữ liệu X không thể được truy xuất bởi các thao tác dữ liệu khác. Lock(X) = 1

1. Kĩ thuật khóa.

- **Hai thao tác trong khóa nhị phân**
  - Lock_item(X)
  - Unlock_item(X)
- Khi một giao dịch muốn truy xuất X, trước tiên nó thực hiện một thao tác Lock_item(X).
  - Nếu Lock(X) = 1, giao dịch phải đợi.
  - Nếu Lock(X) = 0, giá trị Lock(X) gán thành 1, giao dịch được thao tác trên X.
- Sau khi hoàn tất những thao tác trên X, giao dịch thực hiện thao tác Unlock_item(X): gán Lock(X) = 0.
- Khi đó, X có thể được truy xuất bởi các giao dịch khác.
- Giai đoạn giữa Lock_item(X) và Unlock_item(X), giao dịch được gọi là đang giữ khóa trên X.
- Chỉ một giao dịch được giữ khóa trên 1 hạng mục dữ liệu.

1. Kĩ thuật khóa.

- **Giải thuật khóa/ mở khóa**

```
lock_item(X):
B:  if LOCK(X) = 0                  (* item is unlocked *)
        then LOCK(X) ← 1            (* lock the item *)
    else
        begin
        wait (until LOCK(X) = 0
            and the lock manager wakes up the transaction);
        go to B
        end;

unlock_item(X):
    LOCK(X) ← 0;                    (* unlock the item *)
    if any transactions are waiting
        then wakeup one of the waiting transactions;
```

1. Kĩ thuật khóa.

- **Thuộc tính và luật của khóa nhị phân**
  - Khóa nhị phân được thực hiện đơn giản bằng cách thêm 1 biến nhị phân ứng với mỗi hạng mục dữ liệu.
  - Mỗi khóa được ghi nhận với 3 thuộc tính cơ bản:

    <Data_item_name, LOCK, Locking_transaction>

    - Tên hạng mục dữ liệu
    - Giá trị khóa LOCK
    - Giao dịch giữ khóa
  - Hệ thống quản lý các khóa trong một bảng, các hạng mục dữ liệu không có trong bảng là không khóa (unlocked).

1. Kĩ thuật khóa.

- **Mỗi giao dịch phải tuân theo các luật:**

  1. Một giao dịch phải thực hiện thao tác Lock_item(X) trước khi thực hiện bất kì thao tác đọc/ghi nào trên X.

  2. Một giao dịch phải Unlock_item(X) sau khi hoàn tất tất cả thao tác đọc/ghi trên X.

  3. Một giao dịch không thực hiện thao tác Lock_item(X) nếu nó đang giữ khóa trên X.

  4. Một giao dịch không thực hiện thao tác Unlock_item(X) trừ khi nó đang giữ khóa trên X.

1. Kĩ thuật khóa.

Khóa Shared / Exclusive

- Khóa nhị phân hạn chế khi dùng cho dữ liệu vì chỉ một giao dịch được giữ khóa trên một hạng mục dữ liệu X.
- Các thao tác đọc cùng hạng mục dữ liệu trên các giao dịch khác nhau là không xung đột → Ta có thể cho phép nhiều giao dịch được truy xuất X nếu các giao dịch đó chỉ đọc X.
- Nếu một giao dịch thực hiện việc ghi lên X, nó được cấp quyền truy xuất riêng (exclusive).
- Shared locks hay còn gọi là read locks: các giao dịch khác được phép đọc.
- Exclusive locks hay còn gọi là write locks: chỉ 1 giao dịch giữ khóa được thao tác.

1. Kĩ thuật khóa.

Khóa Shared / Exclusive

- Ba thao tác trong giao thức khóa này
  - Read_lock(X)
  - Write_lock(X)
  - Unlock(X)
- Giá trị khóa trên X – Lock(X) có 3 trạng thái:
  - Read-locked (Shared-lock)
  - Write-locked (Exclusive-lock)
  - Unlocked
- Mỗi dòng trong bảng khóa lưu 4 thông tin:
  - Tên hạng mục dữ liệu
  - Giá trị của khóa: read-locked / write-locked
  - Số lượng giao dịch đọc
  - Các giao dịch giữ khóa

1. Kĩ thuật khóa.

Khóa Shared / Exclusive

- **Hệ thống phải tuân theo các luật:**

1. Một giao dịch T phải thực hiện thao tác Read_lock(X) hoặc Write_lock(X) trước khi đọc X.
2. T phải thực hiện thao tác Write_lock(X) trước khi ghi X.
3. T phải thực hiện thao tác Unlock(X) sau khi hoàn tất việc đọc, ghi trong T.
4. T không thực hiện Read_lock(X) nếu nó đang giữ khóa read hoặc write trên X.
5. T không thực hiện Write_lock(X) nếu nó đang giữ khóa read hoặc write trên X.
6. T không thực hiện Unlock(X) trừ khi nó đang giữ khóa read hoặc write trên X.

1. Kĩ thuật khóa.

Khóa Shared / Exclusive

- **Thao tác khóa đọc — read_lock(X)**

```
read_lock(X):
B:  if LOCK(X) = "unlocked"
        then begin LOCK(X) ← "read-locked";
                   no_of_reads(X) ← 1
             end
    else if LOCK(X) = "read-locked"
        then no_of_reads(X) ← no_of_reads(X) + 1
    else begin
            wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
            go to B
         end;
```

1. Kĩ thuật khóa.

Khóa Shared / Exclusive

- **Thao tác khóa ghi — write_lock(X)**

```
write_lock(X):
B:  if LOCK(X) = "unlocked"
        then LOCK(X) ← "write-locked"
    else begin
            wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the
 transaction);
            go to B
        end;
```

1. Kĩ thuật khóa.

Khóa Shared / Exclusive

- **Thao tác mở khóa — unlock(X)**

```
unlock(X):
    if LOCK(X) = "write-locked"
        then begin LOCK(X) ← "unlocked";
                   wakeup one of the waiting transactions, if any
             end
    else if LOCK(X) = "read-locked"
        then begin
                no_of_reads(X) ← no_of_reads(X) - 1;
                if no_of_reads(X) = 0
                    then begin LOCK(X) = "unlocked";
                               wakeup one of the waiting transactions, if any
                         end
             end;
```

1. Kĩ thuật khóa.

Khóa Shared / Exclusive

- **Chuyển đổi khóa (Lock conversion)**
  - Nâng cấp (Upgrade) khóa: Nếu giao dịch T là giao dịch duy nhất đang giữ khóa read trên X tại thời điểm thực hiện thao tác Write_lock(X), khóa read này sẽ được nâng cấp thành khóa Write, nếu không giao dịch phải đợi.
  - Hạ cấp (Downgrade) khóa: từ khóa Write chuyển thành khóa read.
  - Khi đó bảng lưu thông tin về khóa phải thay đổi giá trị tương ứng.

2. Khóa 2 pha

2. Khóa 2 pha

- **Kĩ thuật khóa 2 pha (Two-phase Locking)**
  - Về mặt lý thuyết ta thấy rằng kĩ thuật khóa sẽ đảm bảo được tính khả tuần tự.
  - Tuy nhiên trên thực tế, khi thực hiện đúng luật của cơ chế khóa vẫn không đảm bảo hoàn toàn tính khả tuần tự khi điều khiển song hành.

2. Khóa 2 pha

- **Ví dụ minh họa**
- X = 20, Y = 30
  - T1 → T2: X = 50, Y = 80
  - T2 → T1: X = 70, Y = 50
- Câu hỏi đặt ra:
  - Với lịch trình S2 (thứ tự thao tác khác đi): giá trị cuối cùng của X, Y là gì? Lịch trình có đảm bảo tính nhất quán của CSDL? Có khả tuần tự khi sử dụng cơ chế khóa hay không?

T1	T2
Read_lock(Y)	
Read(Y)	
Unlock(Y)	
Write_lock(X)	Read_lock(X)
Read(X)	Read(X)
X=X+Y	Unlock(X)
Write(X)	Write_lock(Y)
Unlock(X)	Read(Y)
	Y=X+Y
	Write(Y)
	Unlock(Y)

2. Khóa 2 pha

- **Nguyên nhân và giải pháp**
  - Nguyên nhân: Y trong T1 và X trong T2 được unlock quá sớm.
  - Thời điểm mở khóa là một yếu tố quan trọng.
  - Do đó để đảm bảo tính khả tuần tự phải áp dụng thêm một số giao thức khác liên quan đến thời điểm khóa và mở khóa cho mỗi giao dịch.
  - Giao thức phổ biến nhất là khóa 2 pha (Two-phase locking).

2. Khóa 2 pha

- **Nguyên tắc khóa 2 pha**
  - Tất cả thao tác khóa phải diễn ra trước thao tác mở khóa đầu tiên trong giao dịch.
  - Một giao dịch được chia làm 2 pha (giai đoạn):
    - Giai đoạn đầu tiên - tạo khóa (expanding/growing phase): các khóa được phép tạo nhưng không khóa nào được phép mở.
    - Giai đoạn mở khóa (shrinking phase): các khóa đang tồn tại được phép mở nhưng không khóa mới nào được tạo.
  - Trong trường hợp cho phép chuyển đổi khóa:
    - Nâng cấp khóa (Read-locked → Write-locked): thực hiện ở giai đoạn tạo khóa.
    - Hạ cấp khóa: giai đoạn mở khóa.

2. Khóa 2 pha

- **Ví dụ: T1 và T2 tuân theo và không tuân theo khóa 2 pha**

T1	T2
read_lock(Y);	read_lock(X);
read_item(Y);	read_item(X);
unlock(Y);	unlock(X);
write_lock(X);	write_lock(Y);
read_item(X);	read_item(Y);
X := X + Y;	Y := X + Y;
write_item(X);	write_item(Y);
unlock(X);	unlock(Y);

Không tuân theo

T1′	T2′
read_lock(Y);	read_lock(X);
read_item(Y);	read_item(X);
write_lock(X);	write_lock(Y);
unlock(Y)	unlock(X)
read_item(X);	read_item(Y);
X := X + Y;	Y := X + Y;
write_item(X);	write_item(Y);
unlock(X);	unlock(Y);

Tuân theo

2. Khóa 2 pha

- **Một số hạn chế của khóa 2 pha**
- Khóa 2 pha có thể giới hạn số lượng giao dịch đồng thời.
- Giao dịch T không thể mở khóa X sau khi X được sử dụng nếu sau đó T phải khóa Y.
  - → Các giao dịch khác sử dụng đến X buộc phải đợi mặc dù có thể T đã hoàn tất thao tác trên X.
- Hoặc là T phải khóa thêm Y mặc dù T chưa sử dụng đến Y để mở được khóa X.
  - → Các giao dịch khác buộc phải đợi khi thao tác trên Y mặc dù Y chưa sử dụng đến.

2. Khóa 2 pha

```
Read_lock(X)
Read(X)
Write_lock(Y)
Unlock(X)      ← chỉ có thể mở sau khi đã khóa Y
Read(Y)
Y=X+Y
Write(Y)
Unlock(Y)
```

- **Đó là cái giá của việc đảm bảo tính khả tuần tự mà không cần kiểm tra mỗi lịch trình.**
- Khóa 2 pha mặc dù đảm bảo được tính khả tuần tự nhưng nó không cho phép tất cả các lịch trình khả tuần tự do một số lịch trình bị hạn chế bởi giao thức.
- Ngoài ra còn xuất hiện 2 vấn đề: Deadlock và Starvation.

2. Khóa 2 pha

- **Một số biến thể của khóa 2 pha**
- Basic 2PL: kĩ thuật cơ bản mô tả ở trên.
- Conservative (thận trọng, bảo thủ) 2PL: giao dịch khóa toàn bộ dữ liệu cần thiết trước khi giao dịch bắt đầu thực thi (read_set và write_set). Nếu có bất kì dữ liệu nào không thể khóa, giao dịch chờ cho đến khi tất cả dữ liệu được mở khóa → không thực tế.
- Strict (chặt chẽ) 2PL: giao dịch không mở khóa bất kì khóa exclusive nào cho đến khi giao dịch kết thúc.
- Rigorous (khắt khe) 2PL: giao dịch không mở bất kì khóa nào cho đến khi giao dịch kết thúc.

3. Deadlock và Starvation

3. Deadlock và Starvation

- **Deadlock**
  - Deadlock xảy ra khi thực hiện 2 hay nhiều giao dịch đồng thời, khi một giao dịch T chờ đợi một dữ liệu bị khóa bởi giao dịch T′ và ngược lại.
- **Lịch trình S3 (ví dụ deadlock):**

T1	T2
Read_lock(Y)	
Read(Y)	Read_lock(X)
Write_lock(X) (đợi)	Read(X)
	Write_lock(Y) (đợi)

3. Deadlock và Starvation

- **Deadlock Prevention Protocols**
  - Conservative 2PL: khóa tất cả hạng mục cần thiết trước khi thực hiện giao dịch.
  - Một số kĩ thuật khác quyết định cách thức xử lý giao dịch liên quan đến deadlock: đợi/hủy bỏ/ưu tiên, hủy giao dịch khác.
  - Một số kĩ thuật sử dụng khái niệm nhãn thời gian của giao dịch (Transaction timestamp).
  - Nhãn thời gian: định danh duy nhất gán cho mỗi giao dịch dựa trên thứ tự bắt đầu của giao dịch.
  - Giao dịch bắt đầu trước (cũ hơn) sẽ có nhãn thời gian nhỏ hơn: T1 bắt đầu trước T2 $\Longrightarrow$ TS(T1) < TS(T2).

3. Deadlock và Starvation

- Giả sử giao dịch Ti muốn khóa một dữ liệu X đã bị khóa bởi một giao dịch Tj khác. Khi đó có 2 luật để ngăn chặn deadlock:

Wait-die	Wound-wait
Nếu TS(Ti) < TS(Tj): Ti được phép đợi	Nếu TS(Ti) < TS(Tj): hủy Tj, khởi động lại Tj với TS(Tj) như cũ
Ngược lại: hủy Ti, khởi động lại Ti với TS(Ti) như cũ	Ngược lại: Ti được phép đợi

- Cả 2 cùng kết thúc với việc hủy bỏ giao dịch mới hơn mà có thể dẫn đến deadlock.
- Cả 2 kĩ thuật này đảm bảo không xuất hiện deadlock (deadlock-free):
  - Wait-die: chỉ đợi giao dịch bắt đầu sau (younger)
  - Wound-wait: chỉ đợi giao dịch bắt đầu trước (older)
- Tuy nhiên cả 2 kĩ thuật có thể dẫn đến tình trạng hủy bỏ và khởi động lại giao dịch một cách không cần thiết trong trường hợp không xuất hiện deadlock thật sự.

3. Deadlock và Starvation

- **Một số luật khác không sử dụng nhãn thời gian**
  - No waiting algorithm: nếu 1 giao dịch không thể khóa, nó được hủy bỏ và khởi động lại sau 1 khoảng thời gian định trước mà không cần kiểm tra deadlock.
  - Cautious waiting algorithm: nếu Tj không đợi một dữ liệu bị khóa nào khác thì Ti được phép đợi; ngược lại hủy bỏ Ti.

3. Deadlock và Starvation

- **Deadlock Detection**
  - Wait-for Graph:
  - Node: là các giao dịch đang thực hiện.
  - Khi Ti đợi để khóa X đang được khóa bởi Tj: tạo cung Ti → Tj.
  - Khi Tj mở khóa X đó, xóa cung Ti → Tj.
  - Deadlock: đồ thị có chu trình.
  - Lựa chọn giao dịch để hủy (victim selection): tránh các giao dịch được bắt đầu lâu, thực hiện nhiều cập nhật mà chọn giao dịch mới bắt đầu và chưa thực hiện nhiều thay đổi.

3. Deadlock và Starvation

- **Timeouts**
  - Một cách thức đơn giản để xử lý deadlock khác là kĩ thuật sử dụng thời gian chờ (Timeout). Khả thi bởi sự đơn giản.
  - Timeouts: nếu một giao dịch thực hiện việc đợi trong thời gian lớn hơn một khoảng thời gian timeouts định sẵn, hệ thống xem như giao dịch gặp deadlock và hủy bỏ nó mà không cần biết deadlock có thực sự xảy ra hay không.
- **Starvation**
  - Khi 1 giao dịch không thể tiếp tục trong một thời gian quá lâu trong khi giao dịch khác vẫn thực hiện bình thường.
  - Xảy ra do cơ chế đợi hoặc do cơ chế chọn giao dịch hủy (victim selection) không công bằng.
- **Một số giải pháp:**
  - First come first served
  - Chấp nhận độ ưu tiên cho các giao dịch nhưng tăng dần độ ưu tiên cho các giao dịch đợi lâu

4. Kĩ thuật nhãn thời gian.

4. Kĩ thuật nhãn thời gian.

- **Kĩ thuật nhãn thời gian (Timestamp Ordering)**
  - Kĩ thuật khóa: lịch trình tương đương với 1 vài lịch trình tuần tự (hợp với giao thức khóa).
  - Kĩ thuật nhãn thời gian: lịch trình tương đương với 1 lịch trình tuần tự cụ thể được xác định dựa trên nhãn thời gian.
  - Điều khiển song hành dựa trên kĩ thuật sắp xếp nhãn thời gian không sử dụng khóa do đó không phát sinh deadlock.

4. Kĩ thuật nhãn thời gian.

- Nhãn thời gian có thể phát sinh bằng nhiều cách:
  - Thứ tự tăng dần: 1, 2, 3… Tuy nhiên phải reset bộ đếm khi không có giao dịch thực hiện trong một khoảng thời gian.
  - Gán nhãn theo ngày giờ hệ thống.
  - Về cơ bản, kĩ thuật này đảm bảo thứ tự truy xuất mỗi hạng mục dữ liệu bởi các chỉ thị xung đột trong lịch trình không vi phạm nhãn thời gian.

4. Kĩ thuật nhãn thời gian.

- Mỗi hạng mục dữ liệu X gắn với 2 giá trị nhãn thời gian:
  - Read_TS(X): nhãn thời gian lớn nhất trong số các nhãn thời gian của các giao dịch đọc thành công X. Read_TS(X) = TS(T) với T là giao dịch mới nhất (youngest) đọc thành công X.
  - Write_TS(X): nhãn thời gian lớn nhất trong số các nhãn thời gian của các giao dịch ghi thành công X. Write_TS(X) = TS(T) với T là giao dịch mới nhất (youngest) ghi thành công X.

4. Kĩ thuật nhãn thời gian.

- **Thứ tự nhãn thời gian cơ bản (Basic Timestamp Ordering)**
  - Khi giao dịch T thực hiện thao tác Read(X) hoặc Write(X), giải thuật so sánh nhãn thời gian của T với Read_TS(X) hoặc Write_TS(X) để đảm bảo thứ tự thực hiện không bị vi phạm.
  - Nếu thứ tự bị vi phạm, giao dịch T phải hủy và gọi lại với một nhãn thời gian mới.
  - Trong tình huống T hủy bỏ và có rollback dữ liệu → giao dịch T1 nếu có sử dụng giá trị được ghi bởi T phải rollback theo. Tương tự nếu T2 sử dụng giá trị ghi bởi T1 cũng rollback theo.

  → Cascading rollback là một vấn đề và cần có một số giao thức đi kèm.

4. Kĩ thuật nhãn thời gian.

- **Giao dịch T thực hiện thao tác Write(X):**
  - Nếu Read_TS(X) > TS(T) hoặc Write_TS(X) > TS(T): hủy bỏ và rollback T vì đã có giao dịch sau T đọc hoặc ghi lên giá trị X trước khi T thực hiện Write(X).
  - Ngược lại thì thực hiện Write(X) và gán giá trị Write_TS(X) = TS(T).
- **Giao dịch T thực hiện thao tác Read(X):**
  - Nếu Write_TS(X) > TS(T): hủy bỏ và rollback vì T cần đọc giá trị X cũ nhưng đã bị ghi lên bởi một giao dịch nào đó sau T.
  - Nếu Write_TS(X) <= TS(T): thực hiện Read(X) và gán Read_TS(X) bằng giá trị lớn hơn trong 2 giá trị TS(T) và Read_TS(X).
  - Khi phát hiện 2 chỉ thị xung đột (xét về mặt thứ tự thực hiện của chỉ thị), giải thuật này hủy bỏ chỉ thị mới hơn bằng cách hủy bỏ giao dịch tương ứng. Do đó đảm bảo tính khả tuần tự.
  - Deadlock không xuất hiện tuy nhiên xuất hiện việc hủy bỏ - khởi tạo lại giao dịch → có khả năng xuất hiện starvation.

4. Kĩ thuật nhãn thời gian.

- **Ví dụ Lịch trình S4:**

T1	T2
Read(X)	
	Write(X)
Write(X)	

- T1 → T2. T1:Read(X) và T2:Write(X) được thực hiện thành công.
- Xét T1:Write(X): so sánh Write_TS(X) với TS(T1) để quyết định thực hiện Write(X) hay rollback.

4. Kĩ thuật nhãn thời gian.

- **Strict Timestamp Ordering**
  - Một biến thể của nhãn thời gian cơ bản đảm bảo tính khả phục hồi và tính khả tuần tự xung đột.
  - Giao dịch T thực hiện thao tác Read(X) hoặc Write(X) mà TS(T) > Write_TS(X) sẽ tạm dừng thao tác đọc/ghi đó lại cho đến khi giao dịch T′ nào đó đã ghi giá trị X kết thúc (nghĩa là TS(T′) = Write_TS(X)).

4. Kĩ thuật nhãn thời gian.

Thomas's Write Rule

- Một biến thể khác của kĩ thuật nhãn thời gian cơ bản không đảm bảo khả tuần tự xung đột của lịch trình nhưng hạn chế sự rollback trong thao tác Write(X):
  - Nếu Read_TS(X) > TS(T): hủy bỏ T, rollback.
  - Nếu Write_TS(X) > TS(T): không thực hiện thao tác Write(X) nhưng vẫn tiếp tục xử lý, **không rollback**.
  - Nếu không xảy ra 2 trường hợp trên: thực hiện thao tác Write(X) và thay đổi Write_TS(X) = TS(T).

5. Kĩ thuật sử dụng nhiều phiên bản.

5. Kĩ thuật sử dụng nhiều phiên bản.

- **Kĩ thuật nhiều phiên bản (Multiversion)**
  - Giữ lại giá trị cũ của dữ liệu khi dữ liệu được cập nhật.
  - Khi một giao dịch muốn truy xuất một dữ liệu, một phiên bản (version) thích hợp (giá trị) của dữ liệu được chọn để đảm bảo tính khả tuần tự.
  - **Ý tưởng:** một số hành động Read không được chấp nhận ở các kĩ thuật khác vẫn có thể thực hiện thông qua phiên bản cũ của dữ liệu.
  - **Bất lợi:** Cần phải lưu trữ nhiều dữ liệu hơn — tuy nhiên thông thường các dữ liệu cũ vẫn cần được lưu trữ cho các mục đích khác như phục hồi (recovery), lịch sử thay đổi của dữ liệu (data history).
- 2 kĩ thuật nhiều phiên bản:
  - Dựa trên thứ tự nhãn thời gian (Timestamp Ordering).
  - Dựa trên khóa 2 pha (2PL).

5. Kĩ thuật sử dụng nhiều phiên bản.

- **Multiversion – Timestamp Ordering**
  - Nhiều phiên bản X1, X2, …, Xk của X được lưu trữ, với mỗi phiên bản Xi:
    - Read_TS(Xi): giá trị Timestamp lớn nhất của giao dịch đọc thành công Xi.
    - Write_TS(Xi): giá trị Timestamp của giao dịch ghi Xi.
  - Mỗi khi giao dịch T thực hiện Write(X), một giá trị X_{k+1} của X được tạo ra với cả Write_TS(X_{k+1}) và Read_TS(X_{k+1}) bằng TS(T).
  - Mỗi khi giao dịch T thực hiện Read(Xi), giá trị của Read_TS(Xi) được gán bằng giá trị lớn hơn (mới hơn) giữa 2 giá trị Read_TS(Xi) hiện tại và TS(T).

5. Kĩ thuật sử dụng nhiều phiên bản.

- **Để đảm bảo tính khả tuần tự phải thỏa 2 luật sau:**
- Nếu T thực hiện Write(X):
  - Nếu Read_TS(Xi) > TS(T) thì hủy bỏ và rollback T. Trong đó phiên bản i của X là phiên bản có giá trị Write_TS(Xi) lớn nhất trong tất cả phiên bản của X (nhưng nhỏ hơn hoặc bằng TS(T)).
  - Ngược lại, tạo phiên bản Xj của X với Read_TS(Xj) = Write_TS(Xj) = TS(T).
- Nếu T thực hiện Read(X):
  - Tìm phiên bản i của X có Write_TS(Xi) cao nhất và nhỏ hơn hoặc bằng TS(T), trả về giá trị Xi và thay đổi Read_TS(Xi) bằng giá trị lớn nhất trong 2 giá trị TS(T) và Read_TS(Xi).
  - Có thể thấy, việc Read(X) luôn thành công.
  - Trong trường hợp Write(X), giao dịch T có thể bị hủy bỏ khi T ghi một phiên bản X mà cần được đọc bởi giao dịch T′ (sau T, có timestamp là Read_TS(Xi)) nhưng T′ đã đọc phiên bản Xi được ghi bởi giao dịch có timestamp Write_TS(Xi) (không phải T).

5. Kĩ thuật sử dụng nhiều phiên bản.

- Điều kiện chọn phiên bản i khi T thực hiện Write(X):
  - Write_TS(Xi) <= TS(T)
  - Write_TS(Xi) lớn nhất trong số các phiên bản phù hợp điều kiện 1 (sau cùng)
- Nếu Read_TS(Xi) > TS(T): rollback

5. Kĩ thuật sử dụng nhiều phiên bản.

Multiversion – 2PL (khóa 2 pha đa phiên bản)

- Trong mô hình khóa tiêu chuẩn: khóa đọc (read lock/shared lock) và khóa ghi (write lock/exclusive lock).
- 3 trạng thái khóa của một dữ liệu: Read-locked, Write-locked, Certify-locked.

- Bảng tương thích khóa tiêu chuẩn (ngang: giao dịch giữ khóa, dọc: giao dịch yêu cầu khóa):

	Read	**Write**
Read	yes	no
Write	no	no

5. Kĩ thuật sử dụng nhiều phiên bản.

- **Ý tưởng:** Cho phép giao dịch T′ đọc giá trị X trong khi X đang ở trạng thái khóa ghi bởi giao dịch T.
- **Thực hiện:** cho phép 2 phiên bản của X.
  - Một phiên bản X chỉ được ghi khi giao dịch hoàn tất.
  - Một phiên bản X′ được tạo ra khi có giao dịch T giữ khóa ghi trên X.
  - Các giao dịch có thể tiếp tục đọc giá trị X.
  - Giao dịch T đang giữ khóa trên X có thể tiếp tục ghi các giá trị X′ khi cần.
- Khi T muốn hoàn tất (commit), T phải đặt một khóa certify trên tất cả các giá trị mà T đang giữ khóa ghi.
- T phải đợi cho đến khi tất cả các giá trị đó được mở khóa hoàn toàn bởi các giao dịch đang giữ khóa đọc mới có thể hoàn tất việc đặt khóa Certify.
- Cập nhật X bằng X′, xóa X′ và mở khóa certify.

5. Kĩ thuật sử dụng nhiều phiên bản.

- **Bảng tương thích khóa (Lock compatibility table):**

	Read	Write	Certify
Read	yes	yes	no
Write	yes	no	no
Certify	no	no	no

5. Kĩ thuật sử dụng nhiều phiên bản.

- **Một số kĩ thuật khác**
  - Giao thức dựa trên tính hợp lệ: Validation (Optimistic) Concurrency Control Techniques.
  - Đa hạt: Multiple Granularity Locking.

6. Bài tập

6. Bài tập

- **Bài 1.** Cho hai giao dịch cùng cập nhật số dư điểm thưởng của khách hàng trong hai mục dữ liệu E (điểm thưởng) và H (hạng thành viên):

  T1: đọc E, đọc H, ghi H, ghi E

  T2: đọc H, đọc E, ghi E

1. Thêm các thao tác khóa/mở khóa (lock/unlock) vào T1 và T2 sao cho cả hai đều tuân thủ giao thức Strict 2PL.
2. Với các thao tác khóa đã thêm, hãy xây dựng một lịch xen kẽ hợp lệ (không vi phạm tương thích khóa) của T1 và T2.
3. Chỉ ra thời điểm mà một trong hai giao dịch buộc phải chờ, và giải thích tại sao Strict 2PL đảm bảo tính chặt (strict) cho lịch thu được.

6. Bài tập

- **Bài 2.** Ba giao dịch T1, T2, T3 đang chạy đồng thời trên hệ thống quản lý đặt phòng khách sạn, thao tác trên các mục dữ liệu Phòng101, Phòng102, Phòng103. Tại một thời điểm, trạng thái chờ khóa như sau:

  T1 đang giữ khóa ghi trên Phòng101 và đang chờ khóa đọc trên Phòng102 (bị T2 giữ khóa ghi).

  T2 đang giữ khóa ghi trên Phòng102 và đang chờ khóa ghi trên Phòng103 (bị T3 giữ khóa đọc).

  T3 đang giữ khóa đọc trên Phòng103 và đang chờ khóa ghi trên Phòng101 (bị T1 giữ khóa ghi).

a) Vẽ đồ thị chờ (wait-for graph) cho tình huống trên.
b) Hệ thống có đang ở trạng thái bế tắc hay không? Giải thích dựa vào đồ thị.
c) Nếu áp dụng chiến lược wound-wait với thời điểm bắt đầu T1 < T2 < T3 (T1 khởi động sớm nhất), giao dịch nào sẽ bị hủy để giải quyết tình huống? Trình bày rõ quy tắc bạn áp dụng.

6. Bài tập

- **Bài 3.** Cho X = 10, Y = 20. Hai giao dịch T3, T4 thực hiện: T3 tính X := X + Y; T4 tính Y := X + Y. Xét lịch trình S:

a) T3 có tuân theo giao thức khóa 2 pha hay không? Chỉ rõ pha growing và pha shrinking của T3 (nếu có).
b) Tính giá trị cuối cùng của X và Y theo lịch trình S.
c) Đề xuất cách sắp xếp lại thao tác khóa/mở khóa của T3 (không đổi thứ tự đọc/ghi dữ liệu) để lịch trình tuân theo khóa 2 pha, và cho biết giá trị X, Y có thay đổi hay không.

T3	T4
read_lock(Y)	
read_item(Y)	
unlock(Y)	
write_lock(X)	read_lock(X)
read_item(X)	read_item(X)
X := X + Y	unlock(X)
write_item(X)	write_lock(Y)
unlock(X)	read_item(Y)
	Y := X + Y
	write_item(Y)
	unlock(Y)

6. Bài tập

- **Bài 4.**
  - Cho ba giao dịch với nhãn thời gian: TS(T1) = 15, TS(T2) = 25, TS(T3) = 10 (giá trị càng nhỏ nghĩa là giao dịch bắt đầu càng sớm/càng "già"). Xét các tình huống xung đột khóa sau (Ti muốn khóa dữ liệu đang bị khóa bởi Tj):

Tình huống	Ti (muốn khóa)	Tj (đang giữ khóa)
1	T1 (TS=15)	T2 (TS=25)
2	T2 (TS=25)	T1 (TS=15)
3	T3 (TS=10)	T1 (TS=15)

a) Với mỗi tình huống, cho biết kết quả áp dụng giao thức Wait-die.

b) b) Với mỗi tình huống, cho biết kết quả áp dụng giao thức Wound-wait.

c) c) Trong tình huống 3, nếu T3 bị hủy và khởi động lại, nhãn thời gian mới của T3 có được giữ nguyên TS = 10 hay không? Vì sao điều này quan trọng?

6. Bài tập

- **Bài 5.**
  - Cho TS(T5) = 20, TS(T6) = 30. Trạng thái ban đầu: Read_TS(X) = 0, Write_TS(X) = 0. Các thao tác xảy ra theo đúng thứ tự sau:

Bước	Thao tác	TS của giao dịch
1	T6: Write(X)	30
2	T5: Read(X)	20
3	T5: Write(X)	20

a) Xét từng bước, áp dụng đúng 2 luật của Basic Timestamp Ordering (cho Write(X) và Read(X)) để xác định thao tác được thực hiện hay giao dịch bị rollback. Cập nhật Read_TS(X)/Write_TS(X) sau mỗi bước thành công.

b) Giải thích vì sao kết quả ở bước 3 lại xảy ra, liên hệ với nguyên nhân gây cascading rollback.

Tài liệu tham khảo

- Raghu Ramarkrishnan, Johannes Gehrke. *Database Management System*. 3rd Edition, Mc Graw Hill, 2003.
- Tutorials Point, Database Management System, www.tutorialspoint.com, 2015.
- Hector Garcia-Molina, Jeffrey D. Ullman and Jennifer Widom. *Database Systems, the Complete Book*, 2nd Edition, Prentice Hall, 2009
- Ramez Elmasri, Shamkant B. Navathe. *Fundamentals of Database Systems*. *7th Edition*, Pearson, 2015
- Silberschatz,Korth and Sudarshan, Database System Concepts (6th Edition), McGraw-Hill, 2010.

Hết!
Cảm ơn Anh/ Chị đã theo dõi!

GVHD: TS. Phan Thị Thể

FIT – HCM-UTE

Email: thept@hcmute.edu.vn